

DSN4091 · CAPSTONE PROJECT PHASE-I

llmctl

A unified terminal dashboard for multi-provider LLM configuration and diagnostics

SUBMITTED BY Team No. 260

Ayush Kushwah · 23BCE11135

Dolmaa Sharma · 23BCE10761

Dhairya Dani · 23BCE10900

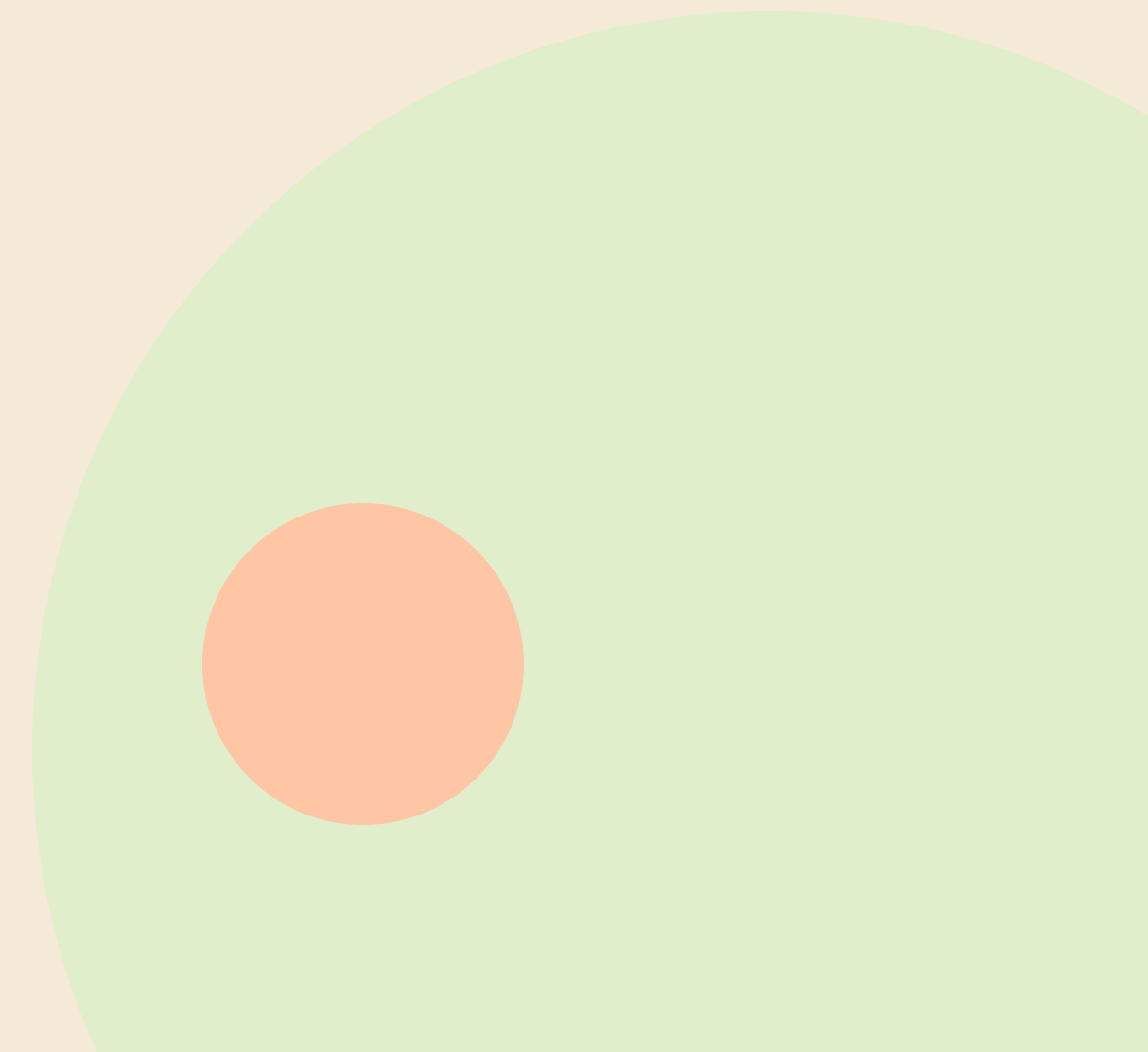
Sanket Subhralok Mohapatra · 23BCE11365

Nivedya B · 23BCE10176

Supervisor — Dr. Vandana Shakya

VIT Bhopal University, Madhya Pradesh

September 2026



Contents

1 **Introduction**

3 **Front End, Back End and
Requirements**

5 **Conclusion**

2 **Existing Work**

4 **Methodology and Results**

Ref **References**

Problem Context

The LLM ecosystem has fragmented. A single development session may draw on three classes of provider, each chosen for a different property.



Frontier reasoning

Architectural decisions and refactoring, routed to hosted frontier models.



Cost-effective volume

Bulk generation and iteration, routed through aggregating gateways.



Local and offline

Privacy-sensitive, latency-bound or disconnected work on local runtimes.

Provider credentials, model identifiers and shell environment variables remain manual at every switch, and the active conversation is interrupted each time.

Motivation

01

Provider plurality

Providers differ in reasoning ability, price, latency and offline support, so the developer moves between them by task rather than settling on one.

02

Context continuity

Decisions, constraints, attempted approaches and open questions are not in the latest message. A fresh conversation discards them; a full replay pays for them again.

03

Local environment fragility

Failures originate in shell configuration, WSL state, credentials or a stalled local service as often as in the provider itself.

Objectives and Phase-I Scope

DELIVERED IN PHASE-I

- A terminal interface for provider and model selection
- Shell-aware configuration for PowerShell and WSL/Bash
- Persistent local sessions, attributed messages and structured notes
- Distillation-based handoff with a pre-switch cost comparison
- A persistent diagnostics panel for WSL, Redis, network and authentication

DEFERRED BY DESIGN

Semantic retrieval and embeddings

Hosted synchronisation and user accounts

Team collaboration features

Broad provider expansion beyond three adapters

The storage schema is structured so these remain additive, not a redesign.

Prior Art Across Four Domains

2.1 · Gateways and API proxies

LiteLLM, Portkey, OpenRouter

Unify request formats across providers, but run as HTTP or cloud services: no terminal interface, no shell propagation, and every request treated independently.

2.2 · CLI tools and single-provider clients

llm, Aider, coding agents

Stateless queries on one side, rich single-provider sessions on the other. Switching means ending the session or copying summaries by hand.

2.3 · Context and memory architectures

Replay, truncation, RAG

Each paradigm trades cost against fidelity; vector retrieval adds latency and an embedding service to a local workflow.

2.4 · Diagnostics and TUI frameworks

Bubble Tea, lazygit, k9s

Deterministic Elm-architecture interfaces are proven for git and Kubernetes; nothing equivalent monitors the local AI development stack.

Comparative Synthesis of Existing Tools

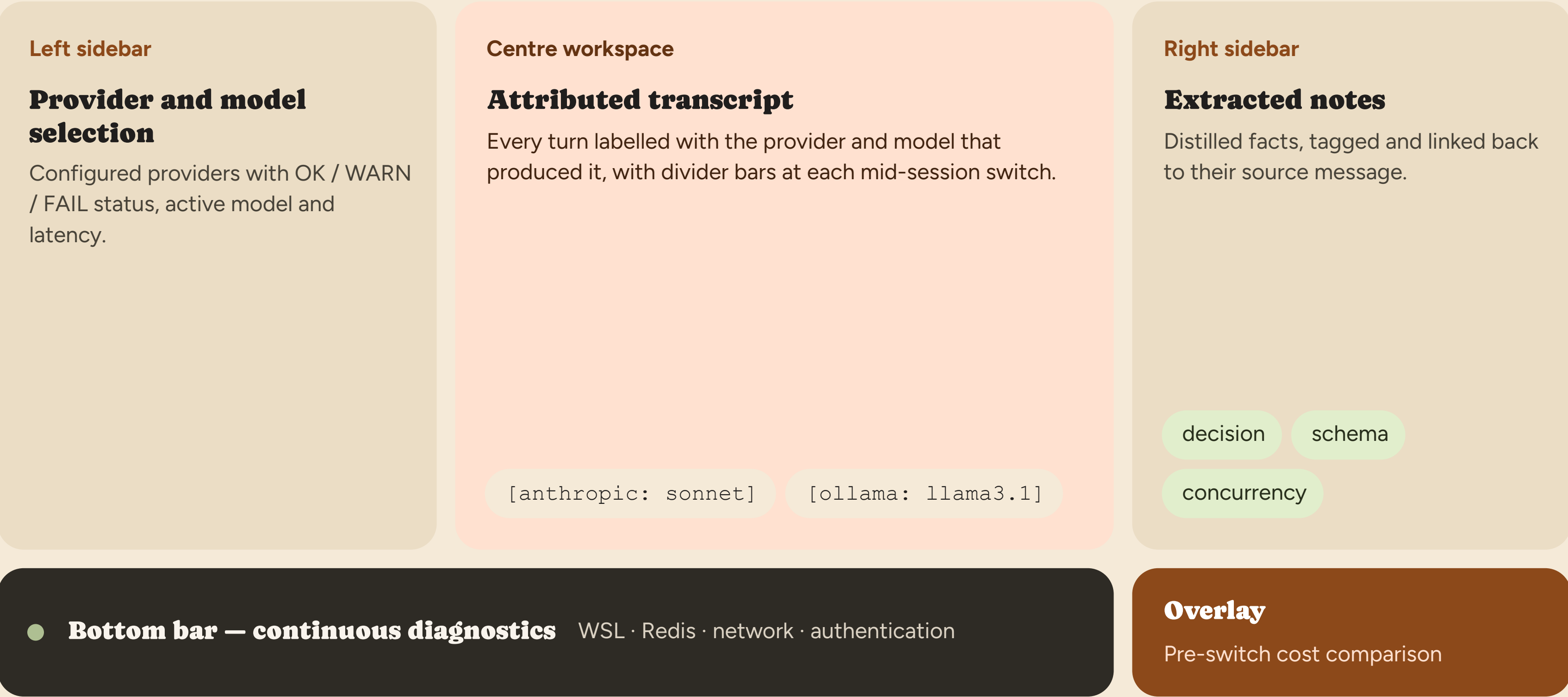
● supported

— absent

Tool / system	Local-first TUI	Win/WSL shell sync	Live diagnostics	Mid-session handoff	Token cost comparison
Willison's llm CLI	— CLI only	—	—	—	—
LiteLLM proxy	— HTTP proxy	—	—	—	—
OpenRouter · Portkey	— cloud SDK	—	—	—	—
Aider · agent clients	● TUI	—	—	— single provider	—
llmctl (proposed)	● Bubble Tea	● auto-export	● Doctor	● distillation	● pre-switch modal

No surveyed tool combines a local-first Go dashboard, cross-shell variable propagation, continuous local diagnostics and distillation-based handoff with pre-switch cost visibility. That absence is the rationale for llmctl.

Terminal Interface Layout



Back-End Subsystems and Schema

<code>internal/shell</code>	Shell detection and native variable export
<code>internal/provider</code>	Unified adapter interface, error classification, token estimation
<code>internal/notes</code>	Asynchronous fact extraction and handoff assembly
<code>internal/doctor</code>	Concurrent health checks under a strict per-check timeout
<code>internal/storage</code>	CGO-free SQLite persistence and encrypted configuration

LLMCTL.DB

sessions — UUID, title, active provider and model

messages — sequence, role, content, token counts, attribution

notes — distilled facts with links_to and superseded_by

switch_events — source and target model, estimated cost breakdown

providers — non-secret display names, base URLs, defaults

Relational from the outset, so vector search can be added without restructuring.

Requirements and Prerequisites

FUNCTIONAL GROUPS

FR-1	Provider management, validation and key encryption
FR-2	One-action switching with shell-native export
FR-3	Continuous note extraction, attribution, cost modal
FR-4	Diagnostics, plus a non-interactive doctor subcommand
FR-5	Scrollable session UI and Markdown / JSON export

NON-FUNCTIONAL LIMITS

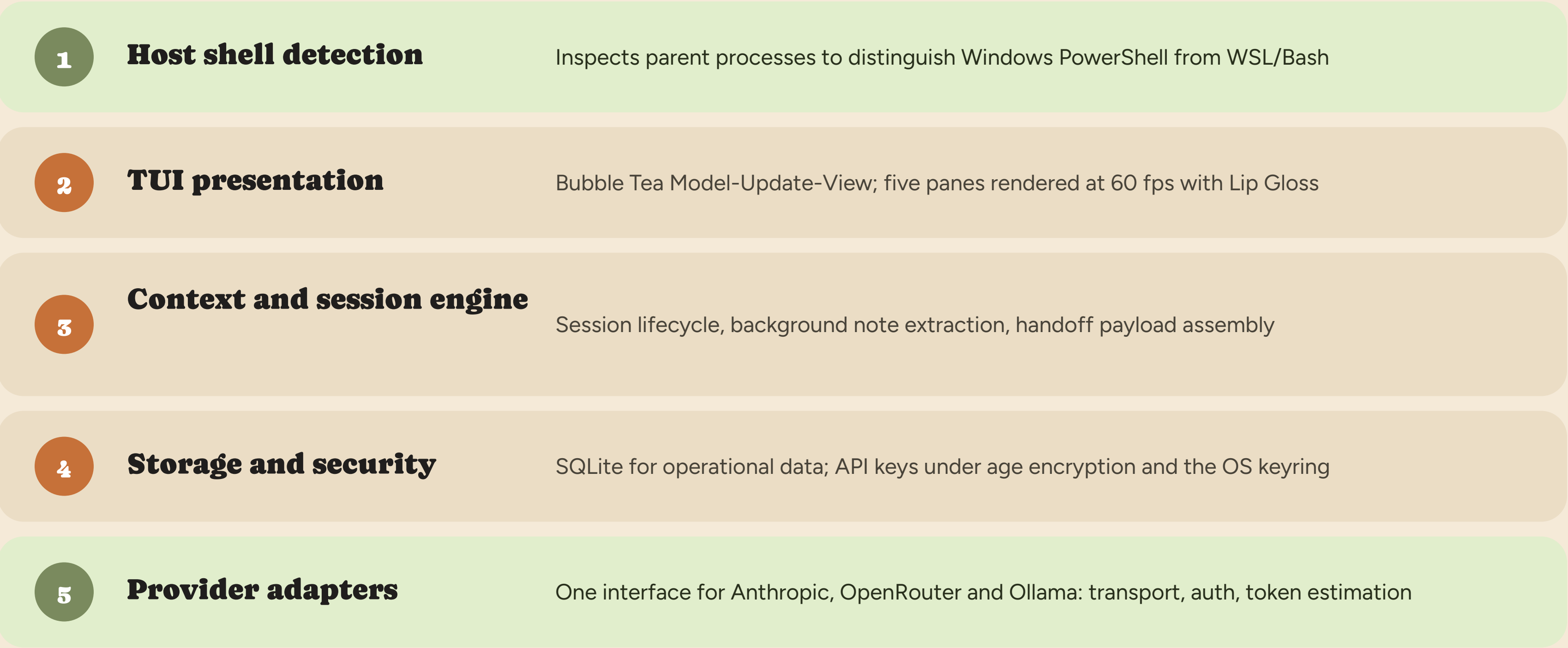
< 100 ms input latency	2.0 s per-check timeout
age + keyring keys encrypted at rest	1 binary static, zero CGO

PREREQUISITES

Windows 10/11 build 19041+ · WSL2 with Ubuntu 22.04 LTS · Go 1.22+ · a true-colour terminal emulator · embedded SQLite · Ollama optional on localhost:11434 · Anthropic or OpenRouter key

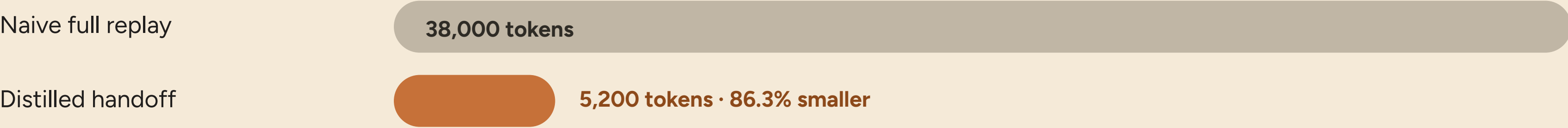
Five-Layer System Architecture

Single static Go binary · no hosted backend



Distillation-Based Context Handoff

Anthropic → Ollama, 30-turn session



Asynchronous Diagnostics Engine

NFR-2 · 2.0 s timeout per check

Doctor runner

Dispatches every check as a concurrent goroutine and collects results without blocking the UI thread.

●	wsl_check	Instance state and inter-process sockets
●	redis_check	Local port binding for supporting services
●	network_check	Reachability of provider endpoints
●	auth_check	Credential validity per configured provider

Results surface as OK, WARN or FAIL in the persistent status bar; a timeout returns a warning rather than stalling the interface.

Token Consumption Across Provider Switches

Bar length = measured saving

Session length	Naive full replay	Distillation handoff	Measured saving
5 turns	3,200 tokens	1,100 tokens	65.6%
15 turns	14,500 tokens	2,800 tokens	80.7%
30 turns	38,000 tokens	5,200 tokens	86.3%
50 turns	82,500 tokens	7,900 tokens	90.4%

Benchmarks simulated multi-turn development sessions switching between frontier models on a Windows 11 host running WSL2.

Latency, Overhead and Footprint

14.2_{ms}

Keyboard input latency

Measured under concurrent note extraction and diagnostics; NFR-1 allows 100 ms.

1.5_s

Note extraction, asynchronous

Completed off the input path, so the transcript stays interactive throughout.

14.2_{MB}

Compiled binary footprint

One static file with embedded migrations and no CGO dependency.

Outcomes and Phase-II Direction

Cross-shell synchronisation

Shell detection and native variable injection remove the manual step between PowerShell and WSL.

Continuous stack visibility

Non-blocking checks report WSL, port, network and authentication health in place.

Cost-aware handoff

Attributed notes retain technical context while removing more than 86% of switch tokens on long sessions.

Security and durability

age encryption with OS keyring integration, and incremental SQLite commits per turn.

PHASE-II

Cross-platform compilation to macOS and Linux, and vector embedding over the existing notes schema — both additive, since OS-specific code is isolated and notes already carry relational attributes.

References

- [1] Berri AI, "LiteLLM: open-source proxy and gateway for multi-provider LLM APIs," 2023.
- [2] S. Willison, "llm: a command-line utility and Python library for LLMs," 2023.
- [3] Charm CLI Team, "Bubble Tea: a TUI framework for Go based on the Elm architecture," 2022.
- [4] R. D. Hipp, "SQLite: design and implementation of an embedded relational database engine," 2020.
- [5] F. Valsorda, "age: a simple, modern and secure file encryption tool," 2019.
- [6] C. Packer et al., "MemGPT: towards LLMs as operating systems," arXiv:2310.08560, 2023.
- [7] H. Jiang et al., "LLMLingua: compressing prompts for accelerated inference," EMNLP 2023.
- [8] Anthropic, "The Claude 3 model family," technical documentation, 2024.
- [9] OpenRouter, "Unified API routing infrastructure and model selection," 2023.
- [10] Ollama, "Local offline LLM inference engine," project documentation, 2024.

Thank you

Questions welcome · Supervisor: Dr. Vandana Shakya